

Lab3: 基于视觉语言模型的视觉问答系统

赵楷越 522031910213

2025 年 5 月 14 日

1 DocVQA

1.1 baseline及问题分析

首先、运行baseline、得到88%的正确率。修改代码使得能够输出其中回答错误的项、一共有以下几类典型错误:

Figure 1: 标点符号及空格错误

```
"question": "What the location address of NSDA?",  
"generated_answer": "1128 Sixteenth St., N.W., Washington, D.C. 20036",  
"correct_answers": [  
  "1128 SIXTEENTH ST., N. W., WASHINGTON, D. C. 20036",  
  "1128 sixteenth st., N. W., washington, D. C. 20036"  
],
```

Figure 2: 回答内容比答案多

```
"question": "What is the name of the company?",  
"generated_answer": "CIGFIL LIMITED, CHENNAI",  
"correct_answers": [  
  "cigfil limited",  
  "CIGFIL LIMITED"  
],
```

Figure 3: 回答内容比答案少

```
"question": "Name the 4 significant personal care brands of ITC?",  
"generated_answer": "Vivel, Superia",  
"correct_answers": [  
  "Essenza Di Wills, Fiama Di Wills, Vivel and Superia"  
],
```

Figure 4: 回答无关信息

```
"question": "What is the name of the ITC Agarbatti brand?",  
"generated_answer": "Aim in Matches",  
"correct_answers": [  
  "Mangaldeep"  
],
```

Figure 5: 数字或字母回答错误

```
"question": "How many rats were fed the control diet?",
"generated_answer": "10",
"correct_answers": [
    "TEN",
    "ten",
    "ten male rats"
],
```

1.2 优化方法尝试

针对上述提到的几类错误、尝试进行图像增强、提示词优化、以及ocr辅助的方式进行优化、并尝试进行不同方法的结合

1.2.1 图像增强

首先进行了图像增强的优化。先把图片强制转化成黑白的；然后大幅提高对比度让文字更清晰；最后进行锐化处理让文字边缘更锋利；同时还用了二值化方法把文字和背景彻底分开。使得llm能够更清晰地分析图片

```
30 def preprocess_image(example):
31     """
32     针对黑白文档的优化预处理:
33     1. 增强对比度突出灰白文字
34     2. 锐化模糊文字边缘
35     3. 自适应二值化处理
36     """
37     image = example["image"].convert('L') # 确保灰度图像
38
39     # 对比度增强 (针对灰白文字)
40     enhancer = ImageEnhance.Contrast(image)
41     image = enhancer.enhance(2.5) # 增强对比度
42
43     # 锐化处理 (针对模糊文字)
44     sharpener = ImageEnhance.Sharpness(image)
45     image = sharpener.enhance(3.0)
46
47     # 自适应阈值二值化
48     image_np = np.array(image)
49     image_np = cv2.adaptiveThreshold(
50         image_np, 255,
51         cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
52         cv2.THRESH_BINARY, 11, 2
53     )
54     return Image.fromarray(image_np)
```

1.2.2 提示词优化

接着、对提示词进行了小幅优化。这里的提示词强调了llm需要”仔细检查文档图像”；加入了”专注于文本内容和布局结构”的具体要求；同时、也把温度参数设为0.5。

```
57 # Enhanced prompt with document-specific instructions
58 prompt = (
59     "You are an expert in document analysis. Carefully examine the document image and "
60     "answer the following question precisely. Focus on textual content, layout structure, "
61     f"Question: {example['question']}\n"
62 )
63
64 chat_response = client.chat.completions.create(
65     model="gpt4o",
66     messages=[
67         {"role": "system", "content": "You are a helpful assistant specialized in image understanding."},
68         {
69             "role": "user",
70             "content": [
71                 {
72                     "type": "image_url",
73                     "image_url": {
74                         "url": base64_image
75                     },
76                 },
77                 {"type": "text", "text": prompt},
78             ],
79         },
80     ],
81     temperature=0.5,
82 )
83 return chat_response.choices[0].message.content
```

1.2.3 OCR辅助

最后、也尝试了OCR辅助进行判断。这里使用了PaddleOCR从图片中提取文字；然后在提示词中嵌入OCR提取的文本，让它结合图文信息回答问题。

```
def extract_text_from_image(image):
    result = ocr.ocr(np.array(image), cls=True)
    texts = []
    for line in result:
        if line: # 检查line是否非空
            for word_info in line: # 遍历每行的识别结果
                if word_info and word_info[1]: # 检查word_info和文本内容是否存在
                    texts.append(str(word_info[1][0])) # 确保转换为字符串
    return " ".join(texts)
```

1.3 结果分析

Table 1: DocVQA不同方法的准确率比较

baseline	图像增强	提示词优化	OCR辅助	图像增强+提示词优化
88%	86%	89%	82%	90%

上表为测试得到的最终结果、发现图像增强以及提示词优化的方法的准确率均在baseline附近、而OCR辅助方法的准确率较低、分析原因可能是OCR提取出的文本依然不具有结构性、而这些非结构化的数据通过提示词的方法输入给llm后反而干扰了llm的回答、导致正确率下降。最终采取了图像增强+提示词优化的混合方案、将DocVQA的实验结果提升到了90%，达到了实验要求。

2 MP-DocVQA

2.1 baseline

首先运行了baseline源程序以及将对应答案页数图片给llm后的结果。得到准确率分别为42%和93%。其中根据对应答案页数answer page idx获取图像的具体代码如下图所示

```
def preprocess_image(example):
    """
    Preprocess the image for better performance
    Args:
        example: dict, the example
    Returns:
        image: Image, the image
    """
    # Choose the first image
    image = example["image_1"]

    # TODO: Choose the true image
    page_idx = example.get("answer_page_idx", 0)
    print(f"使用第 {int(page_idx) + 1} 页图像")
    image_key = f"image_{int(page_idx) + 1}" # 假设页码从1开始
    image = example[image_key]
    # TODO: More methods

    return image
```

2.2 优化方法尝试

参考作业要求、进行了图像合并以及OCR+RAG的方法尝试进行优化

2.2.1 图像合并

首先是图像合并优化。先把文档的所有页面图片统一缩放到相同的宽度并保持比例，保证在20*420*420的图像大小之内；然后把这些处理过的图片从上到下拼接成一张长图给到llm进行问题回答。

```

from PIL import Image

def preprocess_image(example):
    """
    Concatenate all available document pages vertically
    Args:
        example: dict, the example containing multiple images
    Returns:
        concatenated_image: Image, concatenated image
    """
    # Collect all available images
    images = []
    for i in range(1, 21): # MP-DocVQA has up to 20 images
        img_key = f"image_{i}"
        if img_key in example and example[img_key] is not None:
            images.append(example[img_key])

```

2.2.2 OCR+RAG

接着尝试了OCR+RAG的方法。先用OCR把每页文档的文字提取出来；然后计算问题和每页文字的相似度；最后选出最相关的3页合并成一张图发给模型。

```

def preprocess_image(example):
    """
    使用OCR+RAG预处理图像, 选择最相关的图像
    """
    # 获取所有图像键(假设命名为image_1, image_2等)
    image_keys = [key for key in example.keys() if key.startswith('image_')]

    # 把为空的图像键排除
    image_keys = [key for key in image_keys if example[key] is not None]

    # 提取文本和生成嵌入
    texts = []
    embeddings = []
    for key in image_keys:
        image = example[key]
        text = extract_text_from_image(image)
        texts.append(text)
        embeddings.append(embedding_model.encode(text))

```

2.3 结果分析

Table 2: MP-DocVQA不同方法的准确率比较

baseline	correct_page	图像合并	OCR+RAG
42%	93%	68%	81%

上表为测试得到的最终结果、发现图像合并以及OCR+RAG的方法均比baseline，即只使用第一张默认图片的性能更好、其中OCR+RAG的效果和运行效率比图像合并的方法也更好，但仍不及直接将正确页码输入给llm的性能。分析原因可能是OCR+RAG的过程中、仍有可能无法准确地根据问题提取到最相似的文档页图片（和上一个lab类似）。最终实验结果达到了MP-DocVQA的要求。